

Homework 4: Graph

For this assignment you will implement an API for weighted, undirected graphs; then you will use this API to build some small graphs and write a depth-first search.

In `graph.rkt` I've supplied stubs for the code you'll need to write, along with a few suggested helpers and some code to help you with testing.

Orientation

The kinds of graphs we are interested in for this assignment are weighted, undirected graphs whose vertices are natural numbers. In particular, a graph of n vertices will have vertices numbered $0, 1, \dots, n-1$. This makes it straightforward to associate information with each vertex in a vector of size n via direct addressing.

Before defining our signature for weighted, undirected graphs, we define contracts for describing several of the arguments and results involved.

- A vertex is represented as a natural number:

```
let Vertex? = nat?
```

- We use singly-linked lists of vertices, made out of a `cons` struct with `data` and `next` fields as provided by the `cons` library (`import cons`):

```
let VertexList? = Cons.ListC[Vertex?]
```

The `Cons.ListC` contract optionally takes a contract for the list elements, which lets us express that a `VertexList?` is indeed a linked list of `Vertex?`s.

- A weight is a real number (i.e., not an infinity or “Not a Number”), and an optional weight is either a weight or `None`:

```
let Weight? = AndC(num?, NotC(OrC(inf, -inf, nan)))
let OptWeight? = OrC(Weight?, NoneC)
```

Note: this differs slightly from the interfaces we have seen in class.

- A weighted edge is represented by a struct containing two vertices and a weight; we use lists of those as well:

```
struct WEdge:
  let u: Vertex?
  let v: Vertex?
  let w: Weight?
let WEdgeList? = Cons.ListC[WEdge?]
```

Note that `WEdge` is used in the result of one of the graph methods (below), but you don't have to use it internally in your graph representation.

Now we can give our signature for weighted, undirected graphs as a DSSL2 interface with five operations:

```
interface WUGRAPH:
  def n_vertices(self) -> nat?
  def n_edges    (self) -> nat?
  def set_edge(self, u: Vertex?,
               v: Vertex?, w: OptWeight?) -> NoneC
  def get_edge(self, u: Vertex?, v: Vertex?) -> OptWeight?
  def get_adjacent(self, v: Vertex?) -> VertexList?
  def get_all_edges(self) -> WEdgeList?
```

The operations behave as follows:

- The `n_vertices` method returns the number of vertices in the graph: $|V|$.
- The `n_edges` method returns the number of edges in the graph: $|E|$.
- The `set_edge` method adds an edge of weight `w` between vertices `v` and `u` when `w` is a number; if the edge already exists, its weight is updated to `w`. If `w` is `None` then the edge, if it exists, is removed, and if absent remains absent; i.e., this operation is idempotent.

If either `v` or `u` does not exist (e.g., vertex 5 in a 3-vertex graph), `set_edge` should call `error` with a message that includes "out of bounds"..

Note that because the edges of undirected graphs are symmetric, the order of `u` and `v` mustn't matter; this implies that `set_edge` must maintain an invariant.

- The `get_edge` method returns the weight of the edge between vertices `u` and `v` if it exists, or `None` if it does not. `get_edge` has the same error behavior as `set_edge` for out of bounds vertices.
- The `get_adjacent` method returns a list of all vertices that are directly connected to vertex `v`. The order of the list is unspecified.¹
- The `get_all_edges` method returns a list of all edges in the graph, in unspecified order. For each edge in the graph, it includes only one direction in the list. For example, if a graph has an edge of weight 10 between vertices 1 and 3, then the resulting list will contain either `WEdge(1, 3, 10)` or `WEdge(3, 1, 10)`, but not both.

Overall, this interface differs slightly from the ones we have seen in lecture for UU, UD, and WD graphs, both in terms of supported operations and operation behavior. This is intentional: this way, you get to work with multiple graph variants. You are likely to encounter still more variants in practice!

Do note that, just like with the graph interfaces we have seen in class, self-edges are allowed in WUGRAPHS; your implementation should support them.

¹This means any order you like.

Your Task

Representation

Your job is to implement the `WUGraph` class, which must satisfy the `WUGRAPH` interface. To do so, you must choose a representation, as either an adjacency matrix or adjacency lists. Whichever you choose, you will need to add some field(s) to the `WUGraph` class and fill in the `__init__` method to initialize them. **DO NOT** use dictionaries (such as the ones we provide in `hw4-lib.zip`) in your implementation of this class; this is not how we build efficient graph data structures.

1. Define the field(s) for your representation at the top of the `WUGraph` class.
2. Complete the definition of the `__init__` method. The `WUGraph` constructor takes one argument: the number of vertices desired in the new graph.

Note: the representation you use internally doesn't need to correspond to the return types of the ADT operations! You can use whatever kinds of boxes you want, so long as what you create is either an **adjacency matrix** or an **adjacency list**. So long as your operations translate from your internal representation to the types expected by the interface, it's all good. And don't forget: you're welcome to define helper functions and helper structs where they would be helpful!

Graph Operations

Once you've defined your graph representation, you will have to implement the five graph API methods as specified by the `WUGRAPH` interface. Their required time complexities depend on your choice of representation.

Adjacency Matrix Representation

- | | |
|---|---------------------------|
| 3. Implement the <code>n_vertices</code> method. | $\mathcal{O}(1)$ time |
| 4. Implement the <code>n_edges</code> method. | $\mathcal{O}(1)$ time |
| 5. Implement the <code>set_edge</code> method. | $\mathcal{O}(1)$ time |
| 6. Implement the <code>get_edge</code> method. | $\mathcal{O}(1)$ time |
| 7. Implement the <code>get_adjacent</code> method. | $\mathcal{O}(V)$ time |
| 8. Implement the <code>get_all_edges</code> method. | $\mathcal{O}(V ^2)$ time |

Adjacency Lists Representation

The running times of several adjacency list operations depend on d , the maximum degree of the graph.

- | | |
|---|-------------------------------|
| 3. Implement the <code>n_vertices</code> method. | $\mathcal{O}(1)$ time |
| 4. Implement the <code>n_edges</code> method. | $\mathcal{O}(1)$ time |
| 5. Implement the <code>set_edge</code> method. | $\mathcal{O}(d)$ time |
| 6. Implement the <code>get_edge</code> method. | $\mathcal{O}(d)$ time |
| 7. Implement the <code>get_adjacent</code> method. | $\mathcal{O}(d)$ time |
| 8. Implement the <code>get_all_edges</code> method. | $\mathcal{O}(V + E)$ time |

Testing Your Graph

Because the order of the lists returned by `get_adjacent` and `get_all_edges` are unspecified, a test for these methods cannot rely on any particular order as its expected value. Instead, tests for `get_adjacent` and `get_all_edges` should be written to accept *all* possible orders; not just one.

To write tests that accept multiple correct results, we can use a technique called *normalization*. Normalization works by transforming all equivalent values into a single *canonical* value, then having tests compare normalized results against that canonical value.

For example, suppose a graph has a vertex 0 that is adjacent to vertices 1, 2, and 3. Both the lists 1,2,3 and 2,1,3 (and more) are correct results for the `get_adjacent` method. If we normalize by sorting the list of adjacent vertices in increasing order, we can write the following test

```
assert sort(g.get_adjacent()) == cons(1, cons(2, cons(3, None)))
```

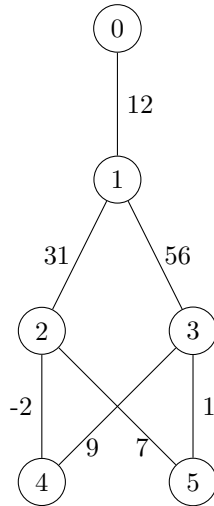
which will accept the results of `get_adjacent` regardless of their order: our (hypothetical) `sort` function would transform 1,2,3, 2,1,3, and all other permutations into the canonical value 1,2,3 which is what the test checks for.

You will need to use normalization when writing your tests for `get_adjacent` and `get_all_edges`. The starter code provides the `normalize_vertices` and `normalize_edges` functions to help you do this.

Building Graphs

The next part of your task is to use the graph class that you built to construct two example graphs: one fixed, and one of your choice. You must construct both these graphs using only methods from the `WUGRAPH` interface. Of course, you're welcome to build more graphs for testing as well!

First, complete the `example_graph` function so it builds the following graph **exactly**:



8. Complete the `example_graph` function.

Second, complete the `my_home_region` function to build a graph representing your hometown and neighboring towns/cities (nodes), road connections between them (edges), and distances between connected cities (weights). Include at least five cities.

Then to associate nodes to cities, build a bidirectional mapping consisting of two dictionaries, one for each direction. Use the *most efficient* dictionary possible for each direction; this may mean a different dictionary for each direction.

Advice: you may want to review the two lectures we had on dictionaries, to make sure you're not overlooking possible choices!

Dependencies

To make sure everyone has access to working dictionary implementations, we have provided you with a compiled solution for homework 3 in `hw4-lib.zip`.

Extract `hw4-lib.zip` in the same directory as your `graph.rkt` file, and the `import` statement at the top will import all the definitions from homework 3, with the interface described in the homework 3 handout.

Note: Be careful to recreate the exact directory structure described above and use the exact import statement we provide. Otherwise your code won't work, either when you try to run it yourself, or when we try to grade it.

Note: The compiled files we provide are specific to Racket version 9.0; if you use any other version, they will not work.

You are welcome to use these dictionaries, or use some of your own creation. If you use one of your own, though, be sure to include its code in your submission; your file must be self-contained (aside from `hw4-lib` and the DSSL2 standard library, of course).

Your `my_home_region` function must return a `CityMap` struct which combines the graph and the two dictionaries. With this bidirectional mapping, we can now do operations in terms of cities, as opposed to having to use nodes directly! Write (at least!) one test that uses the graph and both directions of the bidirectional mapping together to ask a cities-related question of your choice.

In case you would prefer not to share your own hometown, you can use the following city and related towns instead:

- **São Paulo:** biggest city in Brazil
- **Guarulhos:** nearby city/town
- **Mogi das Cruzes:** nearby city/town
- **Osasco:** nearby city/town
- **São Bernardo do Campo:** nearby city/town

For connections and distances, you can consult a map.

9. Complete the `my_home_region` function.
10. Write a test case that uses all the parts of `my_home_region`.

Depth-First Search

Once you have your graph implementation working, there's one more thing to implement, a pre-order depth-first search function to traverse a graph:

```
dfs : WUGRAPH Vertex [Vertex -> None] -> None
```

In class (lecture 9) we used DFS to check reachability between two nodes. However, if we modified our algorithm to not look for a specific target vertex, the DFS algorithm would just end up visiting all vertices reachable from the start; this “graph traversal” is another common application of DFS and is what you are expected to implement here. This function takes a graph `g`, a vertex `start`, and a visitor function `visit`. It performs a depth-first search starting at `start`. As it encounters each vertex `v` for the first time, it calls `visit(v)`. The visitor function is called on each reachable vertex exactly once, in a valid depth-first order.

10. Implement the `dfs` function, which must have the optimal asymptotic time complexity: $\mathcal{O}(|V| + |E|)$ if using adjacency lists, or $\mathcal{O}(|V|^2)$ if using an adjacency matrix.

Keep in mind: your `dfs` function must operate correctly on *any* conforming implementation of the `WUGRAPH` interface; not just yours. So be sure to use only methods which are part of the interface.

Testing DFS

Testing your `dfs` function presents two challenges; or rather, presents two opportunities for learning new testing techniques!

First, `dfs` works indirectly: it does not return any result, and any record of its work happens in the visitor function `f`. Therefore when testing, we need to use a function `f` which gathers enough information for us to check that `dfs` visits the correct vertices in the correct order.

One way to do this is by having `f` construct a *trace* of the visited vertices. A trace is a record of the actions taken by a function or algorithm—in this case: which vertices were visited when—in data structure form which can be compared against an *expected* trace which includes the actions that would be taken by a correct version. To help you do this, we have provided the `dfs_to_list` helper function which returns such traces in linked list form.

Second, because depth-first search could traverse a vertex's neighbors in any order, a given graph may have multiple valid DFS orders. However, not every permutation of the visited vertices is a valid DFS order! Which means that the normalization technique we saw earlier will not work here.

Instead, we must write tests that check for *one of multiple* possible correct values, instead of a specific one. For example, if 0,1,2,3 and 0,2,1,3 are both correct orders for a DFS starting at 0 on a given graph, then a correct test case must accept either order. This can be done by checking that the return value of `dfs_to_list` matches one of the elements of a vector that contains all the correct orders, for example. To make it easier for you to write such tests, we have provided the `one_of` helper function.

Be sure to use both these techniques in your `dfs` tests, to make sure they pass on any correct depth-first search implementation!

Honor Code

Every programming assignment you hand in must begin with the following definition (taken from the Provost's website;² see that for a more detailed explanation of these points):

```
let eight_principles = ["Know your rights.",
    "Acknowledge your sources.",
    "Protect your work.",
    "Avoid suspicion.",
    "Do your own work.",
    "Never falsify a record or permit another person to do so.",
    "Never fabricate data, citations, or experimental results.",
    "Always tell the truth when discussing your work with your instructor."]
```

If the definition is not present, you will receive no credit for the assignment.

Note: Be careful about formatting the above in your source code! Depending on your pdf reader, directly copy-pasting may not yield valid DSSL2 formatting. To avoid surprises, be sure to test your code *after* copying the above definition.

Grading

Please submit your completed version of `graph.rkt`, containing:

- a definition for the `WUGraph` class,
- definitions for the functions `example_graph`, `my_home_region`, and `dfs`,
- sufficient tests to be confident in your code's correctness,
- and the honor code.

Be sure to remove any leftover debugging printing code, comment out any code that would cause infinite loops, and make sure that your submission can run successfully. If your submission produces excessive output, loops infinitely, or crashes, we will not be able to give either you feedback on it or credit for it. **Please run your code one last time before submitting!**

Functional Correctness

We will use four separate test suites to test your submission:

- **Basic WU graph:** construction, `n_vertices`, `get_edge` non-error cases, `get_adjacent`, and `set_edge` in the non-update, non-delete, and non-error cases.
- **Advanced WU graph:** `n_edges`, `get_edge` error cases, `set_edge` update, delete, and error cases, `get_all_edges`.
- **Basic search:** correct DFS traversal order on connected graphs.

²<http://www.northwestern.edu/provost/students/integrity/rules.html>

- **Advanced search:** DFS on disconnected graphs, and stress tests on large random graphs (to check for computational complexity issues).

To get credit for a test suite, your submission must pass *all* its tests.

Non-Functional Correctness

For this assignment, the self-evaluation will be specifically looking for:

- Efficiency from the use of the correct representation and operations
- Maintenance of invariants where relevant
- Good data representation choices

Mutation Testing

We will be using *mutation testing* to assess the thoroughness of your test suites. See the homework 2 handout and the syllabus for more details.

Important: The tests in your test suites *must* accept *any* correct implementation of the homework’s specification; not just yours. In engineering practice, test suites that “overfit” to a given implementation are brittle and thus best avoided: even minor changes that preserve correct behavior are prone to breaking them.

In particular, your tests must rely solely on methods provided by the interface. If a test uses additional methods you added to your class (for debugging reasons, say) beyond those in the interface, that entire test block will be rejected and will not be used when trying to kill mutants. So be sure to stick to the interface and get your tests counted!

For this assignment, your tests for `get_adjacent` and `get_all_edges` must use normalization, as described above. Similarly, your tests for `dfs` must accept all correct depth-first search implementation; see above for how to do so. Failure to do so is likely to lead to these tests getting rejected.

Advice: Keep your test blocks small, and split your testing across multiple test blocks, one for each aspect of the assignment. That way if you made a small mistake in your tests, only the small test block with the mistake will be rejected; as opposed to an entire larger test block that happens to have a mistake in a small part of it.

Stretch Goal

If you’d like a bit of additional practice with graphs, or just want to have a bit of fun with what you built, here’s a small problem you can try to solve.

Create a graph that represents the Chicago “L” network, with multiple train lines and stations for each line. Then write a function that takes a starting station and a natural number n , and returns a list of all the stations you can get to with at most n transfers.

Think carefully about what your nodes and edges should be; the obvious choice may lead to an awkward algorithm, whereas a clever choice may make the algorithm trivial.

Note: This stretch goal is completely optional, and will not be graded. If you do include it in your submission, be careful not to break any of the standard functionality of the assignment (e.g., do not modify the interface, and do not alter operation behavior).